CloudVerse + CloudLab

Installation Guide & 86 Service Requirements — How to Create and Test Each One

A practical, step-by-step manual: how to install and run the whole stack from scratch, followed by 86 concrete requirements spanning both consoles (39 in the CloudVerse DevOps Academy console, 47 in the original CloudLab console) — each one naming exactly what to create, how to create it in the UI with concrete example values, and how to independently verify it actually happened.

Prepared for	Sai Rathish
Organization	Prodapt-AID Practice
Covers	Installation + 86 requirements (39 CloudVerse, 47 CloudLab)
Date	September 11, 2026

Real-backed	a genuine backend call happens — a real Docker container, network, Postgres server, or similar is created.
Simulated-backed	client-side only, for teaching/visualization — no real infrastructure is touched or billed.

1. Installing and running the application

CloudVerse and CloudLab are one npm-workspace monorepo with three long-running processes (a Fastify API + the original static console on :4000, a BullMQ worker, and the CloudVerse Next.js console on :3001) sitting in front of real local infrastructure: real PostgreSQL, real Redis, and a real Docker daemon. Nothing here is mocked — every resource type below is either genuinely provisioned on your machine, or, where noted "Simulated", is honestly labeled as a teaching-only client-side exercise.

1.1 Prerequisites

1. Node.js 20+ and npm.
2. PostgreSQL (any recent version) reachable at the DATABASE_URL you configure.
3. Redis reachable at the REDIS_URL you configure.
4. A running Docker daemon reachable at DOCKER_SOCKET (/var/run/docker.sock by default). Docker's own containerd must already be running before dockerd will start — if dockerd fails to connect, start containerd first.
5. A Go toolchain — needed once, to compile the five small local-only container images described below.

1.2 One-time setup

1. Unzip the application source and, from its root folder, install dependencies for every workspace (apps/api, apps/worker, apps/console, packages/shared) in one command.
2. Copy the two example environment files to their real names and fill in your own values.
3. Generate your own Secrets-Manager master key rather than leaving it blank or reusing an example.
4. Create the Postgres database and run migrations.
5. Build the five local container images the platform provisions from — each is a small Go binary compiled from source and assembled into an image with zero external registry pulls.

```
npm install

cp .env.example .env
cp apps/console/.env.local.example apps/console/.env.local
# edit .env if your Postgres/Redis/Docker socket aren't at the defaults

# Generate a real 32-byte base64 key for SECRETS_MASTER_KEY in .env:
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"

createdb cloudlab   # or: psql -c "CREATE DATABASE cloudlab"
npm run db:migrate

# Build the five local-only images (each is FROM scratch, no external pulls):
bash infra/guest-image/build.sh      # backs Compute instances, Cloud Run, MIG members
bash infra/registry-image/build.sh   # backs the Registry / Artifact Registry feature
bash infra/bucket-image/build.sh     # backs Cloud Storage buckets
bash infra/dns-image/build.sh        # backs Cloud DNS / DNS records
bash infra/lb-image/build.sh         # backs Load Balancers
```

1.3 Running it

Three long-running processes, each in its own terminal (or process manager):

```
npm run dev:api      # Fastify API + the original static console, on :4000
npm run dev:worker   # BullMQ worker -- actually provisions/tears down Docker resources

cd apps/console && npm run dev   # CloudVerse DevOps Academy console, on :3001
```

Open `http://localhost:4000` for the original CloudLab console, and `http://localhost:3001` for the CloudVerse DevOps Academy console. Both talk to the same API and the same database — a resource created in one is visible from the other.

> **Troubleshooting: Docker daemon won't start.** dockerd depends on containerd already running and communicating over /run/containerd/containerd.sock. If "service docker start" fails silently, check whether containerd is up first (start it, then start dockerd) rather than assuming Docker itself is broken.
>
> **Troubleshooting: a real VPC network fails to create with "overlaps with network...".** No real IP-address-management/allocator exists yet, so a freshly-suggested CIDR occasionally collides with a leftover network from an earlier session in a long-lived environment. This is a documented, expected limitation, not a bug — just pick (or regenerate) a different CIDR and retry.

2. How to read each requirement below

Every requirement follows the same shape: an ID, the service or feature it covers, whether it's Real-backed or Simulated, a one-line goal, a numbered "How to create it" procedure with concrete example values you can type in verbatim, and a numbered "How to test it" procedure — usually a mix of what to observe in the UI and, where it adds real confidence, an independent out-of-band check (curl, psql, docker ps, iptables) that doesn't rely on the UI's own say-so. A small number carry an extra note flagging a known quirk or gotcha worth knowing about going in.

Part 1 covers the CloudVerse DevOps Academy console (39 requirements). Part 2 covers the original CloudLab console (47 requirements). Work through them in order within each part where a requirement depends on an earlier one's output (e.g. a Subnet needs a VPC) — this is called out explicitly wherever it applies.

Part 1 — CloudVerse DevOps Academy console (:3001) — 39 requirements

Onboarding

The CloudVerse DevOps Academy console at http://localhost:3001 is where you register, and every account gets a starter project automatically.

REQ-CV-01 Account registration — Real-backed

Create a CloudVerse account and reach the authenticated dashboard.

HOW TO CREATE IT

1. Open http://localhost:3001/login.
2. Click the "Create account" tab.
3. Fill in Name, Email, and Password, then submit the form.

HOW TO TEST IT

1. Confirm the browser redirects to the dashboard at "/" without another login prompt.
2. Refresh the page — you should stay signed in (a real JWT is stored, not a session flag).

> Every registration also silently provisions a starter project named "CloudVerse Lab" behind the scenes — a known limitation is that repeating registration/login flows can create more than one such project rather than reusing one; harmless for solo use, but don't be surprised to see duplicates if you script this.

REQ-CV-02 Onboarding tour — Simulated-backed

Dismiss the welcome walkthrough that otherwise blocks clicks on every first page load.

HOW TO CREATE IT

1. On any page (Dashboard, Architecture Builder, etc.), watch for the "Welcome to CloudVerse DevOps Academy" modal.
2. Click the × close button in its top-right corner.

HOW TO TEST IT

1. Confirm the page behind it becomes clickable immediately.
2. Navigate to a different tab and back — the modal should not reappear once dismissed for that session.

Architecture Builder — real-backed resource nodes

Architecture Builder ("/architecture") is a drag-and-drop canvas (React Flow). Fourteen of its node types are wired to CloudLab's real API: dropping one, filling its Config tab, and clicking "Provision to CloudLab" genuinely creates that resource. The rest (Cloud Armor, Cloud Router, Cloud VPN, Dedicated Interconnect, Memorystore/Redis, Cloud Build, Pub/Sub) are teaching-only simulations with no backend call — useful for practicing layout and dependencies, but nothing to "test" for realness.

REQ-CV-03 VPC Network node

Real-backed

Provision a real Docker bridge network from the canvas.

HOW TO CREATE IT

1. Open the resource palette on the left and drag "VPC Network" onto the canvas.
2. Click the new node to open the Inspector panel, then its Config tab.
3. Set Network Name to "prod-vpc" (CIDR is assigned automatically — a real GCP auto-mode VPC has no CIDR field of its own).
4. Click "Provision to CloudLab" in the toolbar.

HOW TO TEST IT

1. Wait for the node's subtitle to change from "Pending" to "Live on CloudLab".
2. Open the Test tab in the Inspector and click "Run test" — it should report a genuine pass against the real network resource.

> A random /16 CIDR is generated server-side since there's no real IPAM yet; on rare occasions it collides with a leftover network from an earlier session. If Provision fails with "overlaps with network...", just click Provision again — a fresh random CIDR is tried each time.

REQ-CV-04 Subnet node

Real-backed

Provision a real sub-range of a VPC's CIDR, wired as a genuine dependency.

HOW TO CREATE IT

1. Drag "Subnet" onto the canvas near your VPC node.
2. Either drag a connection from the VPC node's right-hand handle to the Subnet node's left-hand handle, or open the Subnet's Config tab and set its "VPC Network" resource-ref field to the VPC directly — both set the same real link.
3. Set Subnet Name to "public-subnet-1", Region to "us-central1", and IP CIDR Range to "10.0.1.0/24" (CloudLab automatically grafts this onto your VPC's real first two octets).
4. Click "Provision to CloudLab".

HOW TO TEST IT

1. Confirm the Subnet reaches "Live on CloudLab" only after its VPC does (dependency ordering is real, not cosmetic).
2. Run a real test from its Test tab.

REQ-CV-05 Compute Engine VM node

Real-backed

Provision a real container standing in for a GCE instance, inside a real subnet.

HOW TO CREATE IT

1. Drag "Compute Engine VM" onto the canvas.
2. Wire its "Subnet" field to the Subnet node created above (edge-drag or the Config tab's resource-ref select).
3. Set Instance Name to "web-vm-1" and pick a Machine Type (e.g. e2-medium).
4. Click "Provision to CloudLab".

HOW TO TEST IT

1. Once "Live on CloudLab", open the Test tab and click "Run test" — this runs a real command inside the container and shows real steps.
2. Watch the live network-flow strip in the panel animate for exactly as long as that real call is in flight.

REQ-CV-06 Cloud Storage bucket node

Real-backed

Provision a real object-store bucket with no dependencies.

HOW TO CREATE IT

1. Drag "Cloud Storage" onto the canvas.
2. Set Bucket Name to "app-assets-prod" in its Config tab.
3. Click "Provision to CloudLab".

HOW TO TEST IT

1. Confirm it reaches "Live on CloudLab" without needing any other node ready first.
2. Run a real test — RealTestPanel performs a genuine object PUT/GET round-trip against the bucket and reports the real result.

REQ-CV-07 Cloud SQL node

Real-backed

Provision a real dedicated PostgreSQL server from the canvas.

HOW TO CREATE IT

1. Drag "Cloud SQL" onto the canvas.
2. Set Instance Name to "prod-db" and Database Engine to "PostgreSQL 15".
3. Click "Provision to CloudLab".

HOW TO TEST IT

1. Wait for "Live on CloudLab", then open the Test tab and run a real test — it executes a genuine query against the real server.

REQ-CV-08 Cloud NAT node

Real-backed

Provision a real NAT gateway wired to a VPC.

HOW TO CREATE IT

1. Drag "Cloud NAT" onto the canvas and wire its VPC Network field to your VPC.
2. Set NAT Gateway Name to "prod-nat".
3. Click "Provision to CloudLab".

HOW TO TEST IT

1. Confirm it only reaches "Live on CloudLab" once the VPC is ready.
2. Run a real test from its Test tab.

REQ-CV-09 Cloud DNS node — Real-backed

Provision a real UDP DNS zone/record wired to a VPC.

HOW TO CREATE IT

1. Drag "Cloud DNS" onto the canvas and wire it to your VPC.
2. Set Zone Name to "prod-zone", DNS Name to "web.internal.", and Record Target IP to "10.0.1.10".
3. Click "Provision to CloudLab".

HOW TO TEST IT

1. Run a real test — it performs a genuine DNS query against the real per-network resolver and shows the answer.

REQ-CV-10 Instance Template + Managed Instance Group — Real-backed

Provision a real reusable template and a self-healing group stamped from it.

HOW TO CREATE IT

1. Drag "Instance Template" onto the canvas, name it "web-template", and provision it.
2. Drag "Managed Instance Group" onto the canvas; wire its "Instance Template" field to the template and its "Subnet" field to your subnet.
3. Set MIG Name to "web-mig" and Min Instances to 2.
4. Click "Provision to CloudLab".

HOW TO TEST IT

1. Confirm the MIG reaches "Live on CloudLab" with 2 real member containers.
2. Run a real test — RealTestPanel checks the real live member count against the target size.

REQ-CV-11 Load Balancer node — Real-backed

Provision a real reverse-proxy load balancer, routed at a real backend.

HOW TO CREATE IT

1. Drag "Load Balancer" onto the canvas and wire its VPC Network field to your VPC.
2. In its Backend Targets field, select the Managed Instance Group created above.
3. Set Load Balancer Name to "web-lb" and click "Provision to CloudLab".

HOW TO TEST IT

1. Once ready, open its Config or detail view for a real Public URL — open that URL in a new tab and confirm you get a real response routed to a live backend member.
2. Run a real test from its Test tab.

REQ-CV-12 Cloud Run node

Real-backed

Provision a real scale-to-zero service wired to a VPC.

HOW TO CREATE IT

1. Drag "Cloud Run" onto the canvas and wire it to your VPC.
2. Set Service Name to "checkout-api" and click "Provision to CloudLab".

HOW TO TEST IT

1. Run a real test — this triggers a real cold-start invoke of the container and reports the genuine response.
2. Wait past the idle timeout and run the test again; observe the same real cold start happening a second time.

REQ-CV-13 Composer node

Real-backed

Provision a real scheduled environment, which auto-creates its own backing pipeline.

HOW TO CREATE IT

1. Drag "Cloud Composer" onto the canvas.
2. Set Environment Name to "nightly-etl" and DAG Schedule to "*/5 * * * *".
3. Click "Provision to CloudLab" — this is a real 2-step create: CloudLab first creates a backing pipeline, then the scheduled environment pointing at it.

HOW TO TEST IT

1. Wait 5 minutes and check the environment's real trigger count increments on its own, with no button clicked.
2. Delete the Composer node afterward and confirm its auto-created backing pipeline is cleaned up too, not leaked.

REQ-CV-14 Full-chain provisioning

Real-backed

Provision an entire multi-resource canvas in a single click, exercising CloudLab's real dependency ordering end to end.

HOW TO CREATE IT

1. With several of the nodes above staged (unprovisioned) on one canvas, click "Provision to CloudLab" once at the top level.
2. Watch the "Provisioning results" dialog list every node's outcome as it resolves.

HOW TO TEST IT

1. Confirm resources with no dependencies (VPC, Cloud Storage) go live first, and dependent resources (Subnet, VM, MIG, LB) wait for their real dependency's id before their own real create call fires.
2. Click "Close" on the results dialog, then reopen the live Architecture Diagram and confirm every node shows "Live on CloudLab".

REQ-CV-15 Real test + real delete on a provisioned node

Real-backed

Exercise RealTestPanel and confirm deleting a canvas node also deletes its real CloudLab counterpart.

HOW TO CREATE IT

1. Select any "Live on CloudLab" node and open its Test tab.
2. Click "Run test".

HOW TO TEST IT

1. Confirm the panel shows real step-by-step progress and a genuine pass/fail tone, not a canned animation.
2. Hover the node to reveal its delete (trash) icon and click it.
3. Reopen the old console (:4000) or the GCP Services Console and confirm the underlying resource is genuinely gone — not just removed from the canvas.

Terraform Lab

Terraform Lab ("/terraform-lab") stages GCP-catalog blocks into an apply plan, generates real Terraform HCL for them, and applies/destroys them against CloudLab's real API.

REQ-CV-16 Stage a VPC block

Real-backed

Add a real-backed VPC block to the apply plan.

HOW TO CREATE IT

1. Open Terraform Lab and its "Build & Apply" tab.
2. In the resource catalog, click "VPC Network", then "Add to plan".

HOW TO TEST IT

1. Confirm the block appears in the plan list with a "Pending" state and has not called the API yet (nothing is real until Apply).

REQ-CV-17 Stage a dependent Subnet block

Real-backed

Wire a real dependency between two staged (not-yet-applied) blocks.

HOW TO CREATE IT

1. Click "Subnet" in the catalog; in its "Add to plan" form, set the VPC Network resource-ref field to the VPC block staged above.
2. Click "Add to plan".

HOW TO TEST IT

1. Confirm the plan view shows the Subnet listed after (dependent on) the VPC block.

REQ-CV-18 Stage a dependent VM block — Real-backed

Complete a 3-block real dependency chain: VPC → Subnet → VM.

HOW TO CREATE IT

1. Click "Compute Engine VM" in the catalog; wire its Subnet field to the staged Subnet block.
2. Click "Add to plan".

HOW TO TEST IT

1. Confirm all three blocks are listed in the plan in dependency order.

REQ-CV-19 Apply the plan — Real-backed

Turn the staged plan into real CloudLab resources, watching the live diagram populate.

HOW TO CREATE IT

1. Click "Apply plan".
2. Note the live Architecture Diagram auto-opens.

HOW TO TEST IT

1. Watch each block's row show a real "RUNNING" then "DONE"/"FAILED" state, in dependency order — Apply waits for CloudLab's real confirmation on each block, not just an instant local flip.
2. Wait for the "Apply complete!" banner and confirm the diagram shows every applied resource with a real id.

> A random VPC CIDR collision here is a known, occasional, and expected failure mode in a heavily-reused sandbox — retrying Apply picks a fresh random CIDR.

REQ-CV-20 Real test + persisted history on an applied block — Real-backed

Run a real test from Terraform Lab and confirm its result is saved server-side, not just in memory.

HOW TO CREATE IT

1. On an applied row, click its flask ("Run a real test") icon.
2. Click "Run test".

HOW TO TEST IT

1. Confirm real step-by-step results and a genuine pass/fail tone.
2. Close the panel, navigate away to another page, then come back and reopen the same block's test panel — confirm it shows "Last run <time ago>" with the same result, without re-running anything.

REQ-CV-21 Destroy all — Real-backed

Tear down every applied, real-backed block and confirm CloudLab's real deletion is awaited.

HOW TO CREATE IT

1. Click "Destroy all".

HOW TO TEST IT

1. Watch each block wait for a real "DELETED" confirmation before the UI marks it destroyed (not an optimistic instant flip).
2. Confirm the "Destroy complete!" banner appears, then check the old console (:4000) or GCP Services Console to confirm the resources are genuinely gone.

CI/CD Simulator

The CI/CD Simulator ("/cicd-simulator") has two pipelines: a Platform (Terraform) pipeline sharing Terraform Lab's real apply/destroy engine, and an App Deployment pipeline that walks a full build→test→deploy flow.

REQ-CV-22 Platform pipeline + generated Terraform — Real-backed

Confirm the Platform pipeline tab shows the real generated Terraform HCL for staged blocks.

HOW TO CREATE IT

1. Open the CI/CD Simulator's default (Platform Pipeline) tab and stage a block or two, same as in Terraform Lab.
2. Click the "Pipeline as Code" tab.

HOW TO TEST IT

1. Confirm the HCL shown matches the real staged blocks and their real config — this is the same generator Terraform Lab uses, not a separate mock.

REQ-CV-23 Run Destroy with nothing applied (negative case) — Real-backed

Confirm the platform explains, rather than silently ignoring, a disabled action.

HOW TO CREATE IT

1. On a fresh project with nothing applied, hover the disabled "Run Destroy" button.

HOW TO TEST IT

1. Confirm a tooltip explains inline why it's disabled ("Nothing is applied yet..."), instead of the click doing nothing with no explanation.

REQ-CV-24 App Deployment — CI provider selection Real-backed

Confirm the generated pipeline code changes correctly per selected CI provider.

HOW TO CREATE IT

1. Click the "App Deployment" tab.
2. Open the provider selector and choose "GitHub Actions" (also try Jenkins, Cloud Build, Bitbucket Pipelines).

HOW TO TEST IT

1. Confirm the generated pipeline code is real, syntactically correct YAML/config for the chosen provider, and changes to match your selected Deploy Target (GKE gets kubectl rollout, Cloud Run gets gcloud run deploy, Compute Engine/MIG gets a rolling update).

REQ-CV-25 Run the App Deployment pipeline Real-backed

Run a full Commit → Build → Test → Push → Deploy → Smoke Test pipeline and record a genuine outcome.

HOW TO CREATE IT

1. With a provider and deploy target selected, click "Run Pipeline".

HOW TO TEST IT

1. Watch each stage run in order with real timing; confirm the outcome (pass or a genuine, sometimes-occurring random failure) is recorded honestly in the deployment history, not scripted to always succeed.

REQ-CV-26 Test endpoint after a real deploy Real-backed

Run a genuine post-deploy check against a real-backed deploy target.

HOW TO CREATE IT

1. After a successful run against a real-backed target (GKE, Cloud Run, or a MIG), find that row in the deployment history and click "Test endpoint".

HOW TO TEST IT

1. Confirm the dialog offers a real RealTestPanel check alongside the simulated HTTP check, and that running it performs a genuine call against the real resource.

GCP Services Console

The GCP Services Console ("/gcp-services") is a catalog-driven resource manager: pick a type from the sidebar, create it with a wizard, and (for the 14 real-backed types) it's genuinely provisioned on CloudLab.

REQ-CV-27 Create a real-backed resource via the wizard

Real-backed

Create a Compute Engine VM through the catalog UI rather than the canvas.

HOW TO CREATE IT

1. Open the Console tab and select "Compute Engine VM" from the sidebar.
2. Click "Create VM", fill the wizard's fields (Instance Name "web-vm-1", pick a Region/Zone/Machine Type), and submit.

HOW TO TEST IT

1. Confirm the new resource card shows "Live on CloudLab" once ready.
2. Click "View" on the card and run a real test from its detail sheet's Test tab.

REQ-CV-28 Create a simulated-only resource

Simulated-backed

Confirm a non-real-backed type is honestly labeled as simulated.

HOW TO CREATE IT

1. Select "Pub/Sub" from the sidebar and click "Create Topic".
2. Fill in Topic Name "order-events" and submit.

HOW TO TEST IT

1. Read the empty-state / creation copy — it should plainly say something like "a fully simulated instance... zero real GCP calls or billing".
2. Confirm no corresponding resource appears in the old CloudLab console (:4000) — proving no real API call was actually made.

REQ-CV-29 Resource detail + real test

Real-backed

Use the shared RealTestPanel from a resource's detail sheet.

HOW TO CREATE IT

1. Click "View" on any "Live on CloudLab" resource card.

HOW TO TEST IT

1. Open the Test tab in the detail sheet and click "Run test"; confirm real step output and the live network-flow strip.

REQ-CV-30 Live Architecture Diagram

Real-backed

View a real-time diagram of everything created in this console.

HOW TO CREATE IT

1. Click "View Architecture Diagram" near the top of the Console tab.

HOW TO TEST IT

1. Confirm every real-backed resource you created above appears as a node with its genuine status.

Docker Lab

Docker Lab ("/docker-lab") teaches container build/push against a real backing Artifact Registry.

REQ-CV-31 Build & push for real

Real-backed

Run a genuine docker build/push from the lab's own Dockerfile.

HOW TO CREATE IT

1. Open Docker Lab and click "Build & push for real".

HOW TO TEST IT

1. Confirm it settles to "Live on CloudLab" (success) or a genuine "Real provisioning failed" with the real build-log error text — not a false success if the build genuinely failed (e.g. a blocked base-image pull).

> If your environment has no network access to public registries, a build using a real base image (not FROM scratch) will genuinely fail here — that's expected and correctly reported, not a bug.

REQ-CV-32 Real test on the built image

Real-backed

Verify the pushed image is real and pullable.

HOW TO CREATE IT

1. Once "Live on CloudLab", click "Run test".

HOW TO TEST IT

1. Confirm the test fetches the real build/pull-command information and reports a genuine digest check.

REQ-CV-33 Delete the real image

Real-backed

Confirm teardown is real, not cosmetic.

HOW TO CREATE IT

1. Click "Delete real image".

HOW TO TEST IT

1. Confirm the panel returns to "Not deployed for real yet".
2. Check the old console's Registry tab and confirm the image row is genuinely gone.

Kubernetes Lab

Kubernetes Lab ("/kubernetes-lab") deploys a real multi-container workload behind CloudLab's own reconciliation loop.

REQ-CV-34 Deploy for real

Real-backed

Provision a real cluster and deployment with genuine pod containers.

HOW TO CREATE IT

1 Open Kubernetes Lab and click "Deploy for real".

HOW TO TEST IT

1 Confirm it reaches "Live on CloudLab" and the pod-count fact row states the real number of running pod containers.

REQ-CV-35 Real test on the deployment

Real-backed

Verify real pod containers match the manifest.

HOW TO CREATE IT

1 Click "Run test".

HOW TO TEST IT

1 Confirm the step list includes a genuine "verify real pod containers match" check against the live count.

REQ-CV-36 Scale for real

Real-backed

Watch CloudLab's real reconciliation loop add/remove pod containers.

HOW TO CREATE IT

1 Adjust the replica count and click "Sync to <N>".

HOW TO TEST IT

1 Wait a few seconds and re-run the test, or check the old console's Kubernetes tab — confirm the real pod-container count now matches the new target.

REQ-CV-37 Delete the real deployment

Real-backed

Confirm teardown removes the real containers.

HOW TO CREATE IT

1 Click "Delete real deployment".

HOW TO TEST IT

1 Confirm the panel returns to "Not deployed for real yet" and, independently, that "docker ps" no longer lists those pod containers.

Cross-cutting behavior

Two behaviors span all six real-test surfaces above (Architecture Builder, Terraform Lab, CI/CD Simulator, GCP Services Console, Docker Lab, Kubernetes Lab).

REQ-CV-38 Live network-flow visualization

Real-backed

Confirm a real test's in-flight moment is actually visible, not just its final result.

HOW TO CREATE IT

1. Open any RealTestPanel and click "Run test".

HOW TO TEST IT

1. Watch the flow strip show a "Request in flight…" animated state for exactly as long as the real call is out, then settle to a solid green (pass) or red (fail) line matching the genuine outcome.

REQ-CV-39 Persistent test history

Real-backed

Confirm real test results survive navigation, not just component state.

HOW TO CREATE IT

1. Run a real test on any resource, then navigate to a completely different page.

HOW TO TEST IT

1. Navigate back and reopen that resource's Test tab — confirm it shows "Last run <time ago>" with the same step detail, without re-running the test.

Part 2 — Original CloudLab console (:4000) — 47 requirements

Onboarding

The original CloudLab console at http://localhost:4000 requires an explicit organization and project before anything else can be created.

REQ-CL-01 Account registration

Real-backed

Create a real user, JWT session, and land on org/project setup.

HOW TO CREATE IT

1. Open http://localhost:4000 and click "Register".
2. Fill in Name, Email, and Password, then click "Create account".

HOW TO TEST IT

1. Confirm you land on an organization/project setup screen, not an error.
2. Refresh the page and confirm you remain signed in.

REQ-CL-02 Create an organization

Real-backed

Create the top-level container every project lives under.

HOW TO CREATE IT

1. In the Organization name field, type "acme-org" and click "Create".

HOW TO TEST IT

1. Confirm the new organization appears selected in the top-bar org selector.

REQ-CL-03 Create a project

Real-backed

Create the project every resource below will belong to.

HOW TO CREATE IT

1. In the Project name field, type "acme-proj" and click "Create".

HOW TO TEST IT

1. Confirm the Dashboard tab loads showing "0 networks / 0 instances" for the new, empty project.

Networking

The Networking tab covers VPCs, subnets, firewall rules, DNS, load balancers, and real fault injection.

REQ-CL-04 Create a VPC network

Real-backed

Provision a genuine Docker bridge network.

HOW TO CREATE IT

1. Open the Networking tab.
2. Set Name to "vpc-main" (a suggested CIDR like "172.30.87.0/24" is pre-filled — keep it or edit it).
3. Click "Create network".

HOW TO TEST IT

1. Wait for the status badge to reach READY.
2. Independently confirm with "docker network ls" on the host that a matching bridge network now exists.

> A collision with an existing network's CIDR is a real, occasional failure in a reused sandbox — just pick a different CIDR and retry.

REQ-CL-05 Create a subnet

Real-backed

Create a validated sub-range of the VPC's CIDR.

HOW TO CREATE IT

1. Scroll to the Subnets section.
2. Set Name to "subnet-a", pick your VPC, a Region, and CIDR "172.30.87.0/26" (must fall inside the VPC's own CIDR).
3. Click "Create subnet".

HOW TO TEST IT

1. Confirm it reaches READY and lists the correct parent network.

REQ-CL-06 Create an ALLOW firewall rule

Real-backed

Confirm a real iptables rule is written on the DOCKER-USER chain.

HOW TO CREATE IT

1. Open the Firewall tab.
2. Set Name "allow-http", pick your network, Direction "ingress", Action "ALLOW", Protocol "tcp", Port "80", CIDR "0.0.0.0/0".
3. Click "Create rule".

HOW TO TEST IT

1. Confirm the rule reaches READY.
2. Run "iptables -L DOCKER-USER -n" on the host and confirm a matching ACCEPT rule now exists.

REQ-CL-07 Create a DENY firewall rule and verify it blocks traffic

Real-backed

Confirm a DENY rule genuinely drops packets, not just displays a red badge.

HOW TO CREATE IT

1. Create an instance on the same network first (see Compute below) if you haven't already.
2. Create a rule: Name "deny-icmp", same network, Direction "ingress", Action "DENY", Protocol "all", CIDR "0.0.0.0/0".

HOW TO TEST IT

1. From the host, try to "ping" the instance's assigned IP before and after the rule exists — confirm it stops responding once the DENY rule is READY.

REQ-CL-08 Create a DNS A record

Real-backed

Resolve a hostname against a real per-network DNS server.

HOW TO CREATE IT

1. Open the DNS tab.
2. Set Name "web-a-record", pick your network, Hostname "web.internal", IP "172.30.87.10".
3. Click "Create record".

HOW TO TEST IT

1. Note the record's resolver IP shown in the table.
2. From a container attached to the same network (or via "docker exec"), run "nslookup web.internal <resolver-ip>" and confirm it returns the configured IP.

REQ-CL-09 Create a Load Balancer and attach a backend

Real-backed

Route real HTTP traffic through a genuine reverse-proxy container.

HOW TO CREATE IT

1. Open the Load Balancers tab, set Name "web-lb" and pick your network, then click "Create load balancer".
2. Once READY, use the row's backend selector to pick a READY instance and click "Add backend".

HOW TO TEST IT

1. Open the load balancer's real Public URL in a browser and confirm you get a genuine response, proxied to the live backend instance.

REQ-CL-10 Real fault injection (Blackhole) Real-backed

Confirm the platform can genuinely sever a network's traffic on demand.

HOW TO CREATE IT

1. On a READY network row, click "Blackhole 30s".

HOW TO TEST IT

1. Confirm the network's status flips to DEGRADED with a "blackholed — auto-reverts <time>" note.
2. Try to reach an instance on that network — confirm it's genuinely unreachable for the full 30 seconds, then automatically recovers (or click "Cancel fault" to revert early).

> This is a real iptables DROP on the network's whole bridge, not a per-instance kill — every instance on the network is affected.

Compute

The Compute tab manages real Docker-volume disks and real containers standing in for VMs, with a genuine exec/logs/terminal/stats toolchain.

REQ-CL-11 Create a disk Real-backed

Provision a real named Docker volume.

HOW TO CREATE IT

1. Open the Disks tab, set Name "data-disk-1" and Size "10" GB.
2. Click "Create disk".

HOW TO TEST IT

1. Confirm it reaches READY and run "docker volume ls" to independently confirm a matching volume exists.

REQ-CL-12 Create an instance Real-backed

Provision a real container standing in for a VM, attached to a subnet and disk.

HOW TO CREATE IT

1. Open the Compute tab, set Name "web-01".
2. Pick your network, subnet, a zone matching the subnet's region, a machine type, and the disk created above.
3. Click "Create instance".

HOW TO TEST IT

1. Confirm it reaches READY with a real assigned IP.
2. Run "docker ps" and confirm a matching container is genuinely running.

REQ-CL-13 Stop and Start an instance

Real-backed

Confirm lifecycle actions map to real container start/stop.

HOW TO CREATE IT

1. On the instance row, click "Stop", wait for STOPPED, then click "Start".

HOW TO TEST IT

1. Watch "docker ps -a" show the container's real status change to "Exited" and back to "Up" in step with the UI.

REQ-CL-14 View real container logs

Real-backed

Confirm the logs panel shows genuine container output.

HOW TO CREATE IT

1. Click the instance's name to open its detail view, then its Logs section.

HOW TO TEST IT

1. Compare against "docker logs <container>" on the host and confirm the content matches.

REQ-CL-15 Exec a real command

Real-backed

Run an arbitrary one-off command genuinely inside the container.

HOW TO CREATE IT

1. In the instance detail view's Exec box, type "uname -a" and submit.

HOW TO TEST IT

1. Confirm the output matches what "docker exec <container> uname -a" returns directly.

REQ-CL-16 Open the live terminal

Real-backed

Get a genuine interactive shell over a WebSocket-backed PTY.

HOW TO CREATE IT

1. In the instance detail view, open the Terminal panel.
2. Type a few commands, e.g. "ls", "pwd", "echo hello > test.txt".

HOW TO TEST IT

1. Confirm the output is real and interactive (not canned).
2. Trigger a few unrelated background changes elsewhere in the project (e.g. create another disk) while the terminal stays open, then confirm the same terminal session is still responsive afterward.

REQ-CL-17 Watch live stats

Real-backed

Confirm CPU/memory/network numbers are real and keep updating.

HOW TO CREATE IT

1. In the instance detail view, watch the "Live resource usage" panel for ~10 seconds.

HOW TO TEST IT

1. Compare the CPU number against "docker stats <container>" run directly, and confirm the panel's numbers keep refreshing rather than freezing at "--".

Registry

A real Docker Distribution v2 registry per project — genuine docker build/push/pull.

REQ-CL-18 Create a registry

Real-backed

Provision a real registry HTTP server.

HOW TO CREATE IT

1. Open the Registry tab, set Name "main-registry", click "Create registry".

HOW TO TEST IT

1. Confirm it reaches READY with a real host shown, e.g. "localhost:3xxxx".

REQ-CL-19 Build & push a real image

Real-backed

Run a genuine docker build and push it to the registry.

HOW TO CREATE IT

1. In "Build & push an image", set Name "myapp", Tag "v1", pick the registry, keep the default Dockerfile, and click "Build & push".

HOW TO TEST IT

1. Confirm it reaches READY and shows a real "docker pull" command.

REQ-CL-20 Verify the pull command

Real-backed

Confirm the image is genuinely retrievable.

HOW TO CREATE IT

1. Copy the shown "docker pull localhost:<port>/myapp:v1" command.

HOW TO TEST IT

1. Run it in a shell on the host and confirm it succeeds against the real registry (no external network needed).

Storage

A real HTTP object store per bucket — verifiable with curl.

REQ-CL-21 Create a bucket

Real-backed

Provision a real object-store endpoint.

HOW TO CREATE IT

1. Open the Storage tab, set Name "app-uploads", click "Create bucket".

HOW TO TEST IT

1. Confirm it reaches READY with a real endpoint URL shown.

REQ-CL-22 PUT an object

Real-backed

Write a real object via curl (recommended) or the UI.

HOW TO CREATE IT

1. curl -X PUT <bucket-endpoint>/hello.txt --data "hello from curl"

HOW TO TEST IT

1. curl <bucket-endpoint>/hello.txt and confirm it returns your text.

> The in-app "Try it: upload / list objects" widget calls the bucket's endpoint directly from the browser, which is a different origin/port than the console — the object store doesn't currently send Access-Control-Allow-Origin, so the browser blocks that specific in-app PUT/List with a CORS error. curl (or any non-browser client) is unaffected and is the reliable way to exercise this feature today.

REQ-CL-23 List objects

Real-backed

Confirm the bucket genuinely tracks what's inside it.

HOW TO CREATE IT

1. curl <bucket-endpoint>/

HOW TO TEST IT

1. Confirm the JSON response's "objects" array includes "hello.txt" from the step above.

Databases

A real, dedicated PostgreSQL 16 server per database resource.

REQ-CL-24 Create a database

Real-backed

Provision a genuine standalone Postgres server.

HOW TO CREATE IT

1. Open the Databases tab, set Name "orders-db", click "Create database".

HOW TO TEST IT

1. Wait for READY and note the real connection string shown.

> initdb + postmaster startup takes a little longer than most other resources — allow up to 30-45 seconds before it reaches READY.

REQ-CL-25 Connect with a real psql client

Real-backed

Prove the server is independently reachable, not a UI label.

HOW TO CREATE IT

1. Copy the shown connection string.

HOW TO TEST IT

1. psql "<connection-string>" -c "select 1" and confirm it returns a row — a genuine external Postgres client talking to a genuine server.

Secrets

Real AES-256-GCM envelope encryption, decrypted only on an explicit, audited Reveal.

REQ-CL-26 Create a secret

Real-backed

Store a value that is genuinely encrypted before it touches disk.

HOW TO CREATE IT

1. Open the Secrets tab, set Name "api-key", Value "my-real-value", click "Create secret".

HOW TO TEST IT

1. Confirm the row shows only a byte length and "AES-256-GCM" label — never the plaintext.

REQ-CL-27 Reveal the secret

Real-backed

Confirm a genuine, separate decrypt call returns the original value.

HOW TO CREATE IT

1. Click "Reveal" on the secret's row.

HOW TO TEST IT

1. Confirm the exact original value you typed appears inline.
2. Open the Audit Log tab and confirm a real audit entry was recorded for the reveal action.

Kubernetes

CloudLab's own container-orchestration layer: real containers as nodes and pods, with a genuine ~10s reconciliation loop.

REQ-CL-28 Create a cluster

Real-backed

Provision a real set of node containers.

HOW TO CREATE IT

1. Open the Kubernetes tab, set Name "prod-cluster", Nodes "2", click "Create cluster".

HOW TO TEST IT

1. Confirm it reaches READY, then "docker ps" for containers labeled for this cluster.

REQ-CL-29 Create a deployment

Real-backed

Stamp out real pod containers under the cluster.

HOW TO CREATE IT

1. Set Name "web-frontend", pick the cluster, Replicas "2", click "Create deployment".

HOW TO TEST IT

1. docker ps --filter label=cloudlab.k8sRole=pod and confirm 2 real containers exist for this deployment.

REQ-CL-30 Scale the deployment

Real-backed

Confirm the real reconciliation loop grows/shrinks the pod count.

HOW TO CREATE IT

1. Change the row's replica number to 4 and click "Scale".

HOW TO TEST IT

1. Re-run "docker ps --filter label=cloudlab.k8sRole=pod" and confirm the count is now 4.
2. Kill one of those containers directly with "docker kill" and confirm the reconciliation loop replaces it within ~10 seconds.

REQ-CL-31 Delete deployment and cluster

Real-backed

Confirm teardown removes the real containers.

HOW TO CREATE IT

1. Delete the deployment row, then the cluster row.

HOW TO TEST IT

1. Confirm "docker ps -a" no longer shows the pod or node containers (beyond normal cleanup delay).

CI/CD

A real CI runner: each run gets a fresh container and executes your stages via genuine docker exec, failing fast on the first non-zero exit.

REQ-CL-32 Create a pipeline

Real-backed

Define real, executable build/test stages.

HOW TO CREATE IT

1. Open the CI/CD tab, set Name "build-and-test".
2. In the stages box, enter one "name: command" per line, e.g.:
   build: echo building > /workspace/artifact.txt
   test: grep building /workspace/artifact.txt
3. Click "Create pipeline".

HOW TO TEST IT

1. Confirm it reaches READY with the correct stage count shown.

REQ-CL-33 Run the pipeline

Real-backed

Confirm real per-stage exit codes and output.

HOW TO CREATE IT

1. Click "Run" on the pipeline row.

HOW TO TEST IT

1. Confirm the run row shows each stage's genuine PASSED/FAILED status, real duration in ms, and captured real stdout.
2. Edit a stage to a command that deliberately fails (e.g. "false") and re-run — confirm it fails fast and skips the remaining stages.

Service Accounts

A real machine identity per resource, with a genuine 2048-bit RSA keypair.

REQ-CL-34 Create a service account

Real-backed

Provision a real RSA keypair, encrypted at rest.

HOW TO CREATE IT

1. Open the Service Accounts tab, set Name "CI Runner", Account ID "ci-runner-sa", click "Create service account".

HOW TO TEST IT

1. Confirm it reaches READY with a generated email like "ci-runner-sa@acme-proj.iam.cloudlab".

REQ-CL-35 Download its key

Real-backed

Retrieve the real private key JSON.

HOW TO CREATE IT

1. Click "Download key" on the service account row.

HOW TO TEST IT

1. Confirm a real 2048-bit RSA private key (JSON key-file shape, with a genuine private_key_id) is shown.

Instance Templates & Managed Instance Groups

A template is a stored blueprint; a group stamps out real containers from it and keeps the live count pinned to target size.

REQ-CL-36 Create an instance template

Real-backed

Define a reusable real blueprint.

HOW TO CREATE IT

1. Open the Instance Groups tab, set Name "web-template", click "Create template".

HOW TO TEST IT

1. Confirm it reaches READY.

REQ-CL-37 Create a managed instance group

Real-backed

Stamp real member containers from the template.

HOW TO CREATE IT

1. Set Name "web-mig", pick the template and a network, Target Size "2", click "Create group".

HOW TO TEST IT

1. Confirm the row shows "2/2 live" and "docker ps" lists 2 matching containers.

REQ-CL-38 Resize and watch self-healing

Real-backed

Confirm the reconciler (every 10s) keeps the live count pinned.

HOW TO CREATE IT

1. Change the resize input to 3 and click "Resize".

HOW TO TEST IT

1. Confirm the live count reaches 3.
2. Kill one member container directly with "docker kill" and confirm the group replaces it automatically within ~10 seconds without any UI action.

NAT Gateway, VPC Peering & Shared VPC

Real host-level iptables rules and routing — a NAT gateway genuinely toggles egress, a peering genuinely opens routing between two networks.

REQ-CL-39 Create a NAT gateway

Real-backed

Toggle real internet egress for a network.

HOW TO CREATE IT

1. Open the NAT & Peering tab, set Name "nat-default", pick a network, click "Create NAT gateway".

HOW TO TEST IT

1. Confirm it reaches READY with a real NAT CIDR shown.

REQ-CL-40 Create a VPC peering and a Shared VPC attachment

Real-backed

Open real routing between two networks, and grant real cross-project attach rights.

HOW TO CREATE IT

1. With two READY networks, fill "Create a VPC peering" (Name "peer-a-b", pick Network A and B), click "Create peering".
2. In "Grant a Shared VPC attachment", fill Name, a Service Project ID, and a network, then click "Grant attachment".

HOW TO TEST IT

1. Confirm both rows reach READY.
2. For the peering, confirm an instance on Network A can now reach an instance on Network B by its real IP (previously blocked).

BigQuery

An honest substitute: a genuinely dedicated real PostgreSQL 16 server per dataset with a real SQL query endpoint.

REQ-CL-41 Create a dataset

Real-backed

Provision a real backing database for a BigQuery-style dataset.

HOW TO CREATE IT

1. Open the BigQuery tab, set Name "analytics-ds", click "Create dataset".

HOW TO TEST IT

1. Confirm it reaches READY with a dataset id shown.

REQ-CL-42 Run a real SQL query

Real-backed

Confirm the query box genuinely executes SQL, not a canned response.

HOW TO CREATE IT

1. In the dataset's SQL box, type "SELECT now()" and click "Run query".

HOW TO TEST IT

1. Confirm a real, current timestamp is returned. Try a deliberately malformed query and confirm a real SQL error comes back instead of a silent success.

Composer

Wraps a real CI/CD pipeline with a real cron schedule.

REQ-CL-43 Create a scheduled environment

Real-backed

Confirm a pipeline genuinely auto-runs on a schedule with nobody clicking Run.

HOW TO CREATE IT

1. Create a READY pipeline first (see CI/CD above).
2. Open the Composer tab, set Name "nightly-etl", pick the pipeline, Schedule "*/5 * * * *", click "Create environment".

HOW TO TEST IT

1. Wait 5+ minutes without touching the CI/CD tab, then return here and confirm the Triggers count has incremented on its own.

Cloud Run

Real scale-to-zero: no container exists until the first real Invoke, and an idle checker genuinely stops it after a timeout.

REQ-CL-44 Create a service

Real-backed

Register a scale-to-zero service definition.

HOW TO CREATE IT

1. Open the Cloud Run tab, set Name "hello-service", pick a network, click "Create service".

HOW TO TEST IT

1. Confirm it reaches READY in an IDLE state — with no container running yet.

REQ-CL-45 Invoke it (real cold start)

Real-backed

Prove the container is genuinely created on first request, not pre-warmed.

HOW TO CREATE IT

1. Click "Invoke" on the service row.

HOW TO TEST IT

1. Confirm a real result JSON is returned and the state flips to RUNNING.
2. Wait past the configured idle timeout without invoking again, then confirm the state genuinely returns to IDLE (the container is actually stopped, checked every ~15s).

IAM & Observability

Every API call is audited automatically, and IAM enforces real allow/deny per role.

REQ-CL-46 Review IAM roles and bindings

Real-backed

Confirm role definitions and project bindings are real, enforced data.

HOW TO CREATE IT

1. Open the IAM tab.

HOW TO TEST IT

1. Confirm your own user id appears in Project bindings with the Owner role, and that built-in roles list real allow/deny glob patterns.
2. Register a second account, share the project at a lower role, and confirm an action outside that role's Allow list is genuinely rejected (a real 403), not just hidden in the UI.

REQ-CL-47 Activity feed and Audit Log

Real-backed

Confirm every action you've taken above left a real trail.

HOW TO CREATE IT

1. Open the Activity tab, then the Audit Log tab.

HOW TO TEST IT

1. Pick any request id from the Audit Log and confirm you can find its exact HTTP method/endpoint/status recorded — there is no code path that skips this log.
2. Send a real CORS preflight (OPTIONS) request against any endpoint, e.g. with curl, and confirm the server answers cleanly with no server-side error logged.
